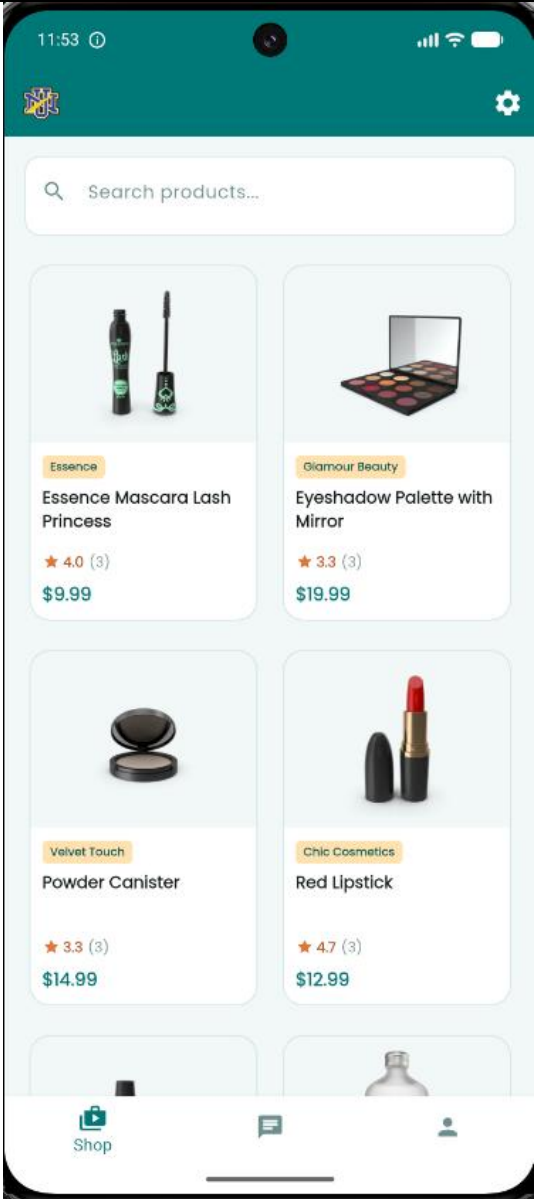


Lab Activity 3

Submission Form

Name	Jamaine Grace Tuazon
Section	INF231
Date	August 20, 2026

Place output based on the source code.



Place your sample code and output based on the enhancement. You may add rows or cells if needed.

Source code – [name of the file]



cart.dart

```
class Cart {
  final int id;
  final List<CartProduct> products;
  final double total;
  final double discountedTotal;
  final int userId;
  final int totalProducts;
  final int totalQuantity;

  Cart({
    required this.id,
    required this.products,
    required this.total,
    required this.discountedTotal,
    required this.userId,
    required this.totalProducts,
    required this.totalQuantity,
  });

  factory Cart.fromJson(Map<String, dynamic> json) {
    return Cart(
      id: json['id'] ?? 0,
      products: (json['products'] as List?)
        ?.map((e) => CartProduct.fromJson(e))
        .toList() ??
        [],
      total: (json['total'] as num?)?.toDouble() ?? 0.0,
      discountedTotal:
        (json['discountedTotal'] as num?)?.toDouble() ?? 0.0,
      userId: json['userId'] ?? 0,
      totalProducts: json['totalProducts'] ?? 0,
      totalQuantity: json['totalQuantity'] ?? 0,
    );
  }

  Map<String, dynamic> toJson() {
    return {
      'id': id,
      'products': products.map((e) => e.toJson()).toList(),
      'total': total,
      'discountedTotal': discountedTotal,
      'userId': userId,
      'totalProducts': totalProducts,
      'totalQuantity': totalQuantity,
    };
  }
}

class CartProduct {
  final int id;
  final String title;
  final double price;
  final double discountPercentage;
  final double discountedTotal;
  final String thumbnail;

  CartProduct({
    required this.id,
    required this.title,
    required this.price,
    required this.quantity,
    required this.total,
    required this.discountPercentage,
    required this.discountedTotal,
    required this.thumbnail,
  });

  factory CartProduct.fromJson(Map<String, dynamic> json) {
    return CartProduct(
      id: json['id'] ?? 0,
      title: json['title'] ?? '',
      price: (json['price'] as num?)?.toDouble() ?? 0.0,
      quantity: json['quantity'] ?? 0,
      total: (json['total'] as num?)?.toDouble() ?? 0.0,
      discountPercentage:
        (json['discountPercentage'] as num?)?.toDouble() ?? 0.0,
      discountedTotal:
        (json['discountedTotal'] as num?)?.toDouble() ?? 0.0,
      thumbnail: json['thumbnail'] ?? '',
    );
  }

  // Saves the new quantity and computes the prices for total
  void updateQuantity(int newQuantity) {
    quantity = newQuantity;
    total = price * newQuantity;
    discountedTotal = total - (total * discountPercentage / 100);
  }

  Map<String, dynamic> toJson() {
    return {
      'id': id,
      'title': title,
      'price': price,
      'quantity': quantity,
      'total': total,
      'discountPercentage': discountPercentage,
      'discountedTotal': discountedTotal,
      'thumbnail': thumbnail,
    };
  }
}
```



cart_screen.dart

```
import 'package:flutter/material.dart';
import 'package:flutter_screenutil/flutter_screenutil.dart';
import '../constants.dart';
import '../models/cart.dart';
import '../services/cart_service.dart';
import '../widgets/custom_text.dart';

// shows the cart of the user
class CartScreen extends StatefulWidget {
  const CartScreen({super.key});

  @override
  State<CartScreen> createState() => _CartScreenState();
}

class _CartScreenState extends State<CartScreen> {
  // talks to the API
  final CartService _cartService = CartService();

  // stores the cart items from the API.
  Cart? _cart;
  // True while waiting sa API
  bool _isLoading = true;
  // error text if the API call failed
  String _errorMessage = '';

  @override
  void initState() {
    super.initState();
    // Gets the cart when the screen opens
    _loadUserCart();
  }

  // renderd the cart that belongs to the current user
  Future<void> _loadUserCart() async {
    setState(() {
      _isLoading = true;
      _errorMessage = '';
    });

    try {
      // request for the carts
      final carts = await _cartService.getCartsByUserId(currentUserId);
      if (!mounted) return;
      setState(() {
        // takes the first cart
        _cart = carts.isNotEmpty ? carts.first : null;
        _isLoading = false;
      });
    } catch (e) {
      if (!mounted) return;
      setState(() {
        _isLoading = false;
        _errorMessage = e.toString();
      });
    }
  }

  // Adds up the quantity of every item
  int get _totalQuantity {
    var count = 0;
    for (final item in _cart!.products) {
      count += item.quantity;
    }
    return count;
  }

  // Adds up the price before discount of every item
  double get _subtotal {
    var sum = 0.0;
    for (final item in _cart!.products) {
      sum += item.total;
    }
    return sum;
  }

  // Adds up the price after discount of every item
  double get _discountedSubtotal {
    var sum = 0.0;
    for (final item in _cart!.products) {
      sum += item.discountedTotal;
    }
    return sum;
  }

  // Runs when the plus or minus button is pressed for quantity increase or decrease
  void _changeQuantity(CartProduct item, int change) {
    final newQuantity = item.quantity + change;
    if (newQuantity < 1) return;
    setState(() {
      item.updateQuantity(newQuantity);
    });
  }

  @override
  Widget build(BuildContext context) {
    final theme = Theme.of(context);
    final isDark = theme.brightness == Brightness.dark;

    return Scaffold(
      backgroundColor: theme.scaffoldBackgroundColor,
      body: SafeArea(
        child: _buildMainContent(theme, isDark),
      ),
    );
  }

  Widget _buildMainContent(ThemeData theme, bool isDark) {
    if (!_isLoading) {
      return _buildSkeletonList(isDark);
    }

    // shows the error and a Try Again button if API call failed
    if (_errorMessage.isNotEmpty) {
      return Center(
        child: Padding(
          padding: EdgeInsets.all(24.r),
          child: Column(
            mainAxisAlignment: MainAxisAlignment.center,
            children: [
              Icon(
                Icons.wifi_off_rounded,
                size: 64.sp,
                color: isDark ? AppColors.darkMuted : AppColors.lightMuted,
              ),
              SizedBox(height: 16.h),
              CustomText(
                text: 'Oops! Something went wrong',
                fontSize: 16.sp,
                fontWeight: FontWeight.bold,
              ),
              SizedBox(height: 8.h),
              CustomText(
                text: _errorMessage,
                fontSize: 13.sp,
                textAlign: TextAlign.center,
              ),
              SizedBox(height: 20.h),
              ElevatedButton.icon(
                onPressed: _loadUserCart,
                icon: const Icon(Icons.refresh),
                label: const Text('Try Again'),
                style: ElevatedButton.styleFrom(
                  shape: RoundedRectangleBorder(
                    borderRadius: BorderRadius.circular(8.r),
                  ),
                ),
              ),
            ],
          ),
        ),
      );
    }
  }
}
```



```
final cart = _cart;

// if user has no cart or the cart has no items
if (cart == null || cart.products.isEmpty) {
  return Center(
    child: Column(
      mainAxisAlignment: MainAxisAlignment.center,
      children: [
        Icon(
          Icons.remove_shopping_cart_outlined,
          size: 64.sp,
          color: isDark ? AppColors.darkMuted : AppColors.lightMuted,
        ),
        SizedBox(height: 16.h),
        CustomText(
          text: 'Your cart is empty',
          fontSize: 16.sp,
          fontWeight: FontWeight.bold,
        ),
      ],
    ),
  );
}

return Column(
  children: [
    // number of cart items
    Padding(
      padding: EdgeInsets.symmetric(horizontal: 16.w, vertical: 12.h),
      child: Row(
        mainAxisAlignment: MainAxisAlignment.end,
        children: [
          CustomText(
            text: '$_totalQuantity items',
            fontSize: 13.sp,
            fontWeight: FontWeight.w600,
            color: isDark ? AppColors.darkMuted : AppColors.lightMuted,
          ),
        ],
      ),
    ),
    // list of products inside the cart.
    Expanded(
      child: RefreshIndicator(
        onRefresh: _loadUserCart,
        child: ListView.separated(
          padding: EdgeInsets.symmetric(horizontal: 16.w, vertical: 4.h),
          itemCount: cart.products.length,
          separatorBuilder: (context, index) => SizedBox(height: 12.h),
          itemBuilder: (context, index) {
            //one card for each product in the cart
            final item = cart.products[index];
            return _buildCartItemCard(context, item, theme, isDark);
          },
        ),
      ),
    ),
    // total
    _buildSummary(cart, theme, isDark),
  ],
);
}
```



```
// opens the product detail screen from the cart item card.
Widget _buildCartItemCard(
  BuildContext context,
  CartProduct item,
  ThemeData theme,
  bool isDark,
) {
  return Card(
    elevation: 0,
    color: theme.cardTheme.color,
    shape: RoundedRectangleBorder(
      borderRadius: BorderRadius.circular(16.r),
      side: BorderSide(
        color: isDark ? AppColors.darkBorder : AppColors.lightBorder,
      ),
    ),
    clipBehavior: Clip.antiAlias,
    child: InkWell(
      onTap: () {
        Navigator.pushNamed(
          context,
          '/product-detail',
          arguments: item.id,
        );
      },
      child: Padding(
        padding: EdgeInsets.all(10.r),
        child: Row(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            /// Product thumbnail
            Container(
              width: 80.w,
              height: 80.h,
              padding: EdgeInsets.all(4.r),
              decoration: BoxDecoration(
                color: isDark
                  ? AppColors.darkBackground
                  : AppColors.lightBackground,
                borderRadius: BorderRadius.circular(12.r),
              ),
              child: Image.network(
                item.thumbnail,
                fit: BoxFit.contain,
                loadingBuilder: (context, child, loadingProgress) {
                  if (loadingProgress == null) return child;
                  return Center(
                    child: CircularProgressIndicator(
                      strokeWidth: 2,
                      value: loadingProgress.expectedTotalBytes != null
                        ? loadingProgress.cumulativeBytesLoaded /
                          loadingProgress.expectedTotalBytes!
                        : null,
                    ),
                  );
                },
              ),
              errorBuilder: (context, error, stackTrace) => Center(
                child: Icon(
                  Icons.broken_image,
                  color:
                    isDark ? AppColors.darkMuted : AppColors.lightMuted,
                ),
              ),
            ),
            SizedBox(width: 12.w),
            /// Item info
            Expanded(
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.start,
                children: [
                  CustomText(
                    text: item.title,
                    fontSize: 14.sp,
                    fontWeight: FontWeight.w600,
                    maxLines: 2,
                    overflow: TextOverflow.ellipsis,
                  ),
                  SizedBox(height: 6.h),
                  Row(
                    children: [
                      CustomText(
                        text: '\${item.price.toStringAsFixed(2)}',
                        fontSize: 13.sp,
                        fontWeight: FontWeight.w500,
                      ),
                      SizedBox(width: 6.w),
                      Icon(
                        Icons.local_offer_outlined,
                        size: 13.sp,
                        color: AppColors.ember,
                      ),
                      SizedBox(width: 4.w),
                      CustomText(
                        text:
                          '\${item.discountPercentage.toStringAsFixed(0)}% off',
                        fontSize: 11.sp,
                        color: isDark ? AppColors.sand : AppColors.emberDeep,
                      ),
                    ],
                  ),
                  SizedBox(height: 6.h),
                  /// The minus button
                  Row(
                    children: [
                      _buildQuantityButton(
                        icon: Icons.remove,
                        theme: theme,
                        isDark: isDark,
                        onPressed: item.quantity > 1
                          ? () => _changeQuantity(item, -1)
                          : null,
                      ),
                      /// Shows the current quantity of this product
                      Padding(
                        padding: EdgeInsets.symmetric(horizontal: 12.w),
                        child: CustomText(
                          text: '${item.quantity}',
                          fontSize: 14.sp,
                          fontWeight: FontWeight.bold,
                        ),
                      ),
                      /// Plus button
                      _buildQuantityButton(
                        icon: Icons.add,
                        theme: theme,
                        isDark: isDark,
                        onPressed: () => _changeQuantity(item, 1),
                      ),
                    ],
                  ),
                ],
              ),
            ),
            Column(
              crossAxisAlignment: CrossAxisAlignment.end,
              children: [
                CustomText(
                  text: '\${item.discountedTotal.toStringAsFixed(2)}',
                  fontSize: 14.sp,
                  fontWeight: FontWeight.bold,
                  color: theme.colorScheme.primary,
                ),
                SizedBox(height: 2.h),
                CustomText(
                  text: '\${item.total.toStringAsFixed(2)}',
                  fontSize: 11.sp,
                  color: isDark ? AppColors.darkMuted : AppColors.lightMuted,
                ),
              ],
            ),
          ],
        ),
      ),
    ),
  );
}
```



```
Widget _buildQuantityButton({
  required IconData icon,
  required ThemeData theme,
  required bool isDark,
  required VoidCallback? onPressed,
}) {
  final isDisabled = onPressed == null;

  return InkWell(
    // opens the product detail screen.
    onTap: onPressed ?? () {},
    borderRadius: BorderRadius.circular(8.r),
    child: Container(
      width: 28.w,
      height: 28.h,
      decoration: BoxDecoration(
        color: isDisabled
          ? (isDark ? AppColors.darkSkeleton : AppColors.lightSkeleton)
          : theme.colorScheme.primaryContainer,
        borderRadius: BorderRadius.circular(8.r),
        border: Border.all(
          color: isDark ? AppColors.darkBorder : AppColors.lightBorder,
        ),
      ),
    ),
    child: Icon(
      icon,
      size: 16.sp,
      color: isDisabled
        ? (isDark ? AppColors.darkMuted : AppColors.lightMuted)
        : theme.colorScheme.onPrimaryContainer,
    ),
  ),
);

}

//total section at the bottom of the cart screen
Widget _buildSummary(Cart cart, ThemeData theme, bool isDark) {
  return Container(
    width: double.infinity,
    padding: EdgeInsets.symmetric(horizontal: 16.w, vertical: 12.h),
    decoration: BoxDecoration(
      color: theme.cardTheme.color,
      border: Border(
        top: BorderSide(
          color: isDark ? AppColors.darkBorder : AppColors.lightBorder,
        ),
      ),
    ),
    child: Column(
      children: [
        // How many different products are in the cart
        Row(
          mainAxisAlignment: MainAxisAlignment.spaceBetween,
          children: [
            CustomText(
              text: 'Total products',
              fontSize: 13.sp,
            ),
            CustomText(
              text: '${cart.totalProducts}',
              fontSize: 13.sp,
              fontWeight: FontWeight.w600,
            ),
          ],
        ),
        SizedBox(height: 4.h),
        // Total price before the discount
        Row(
          mainAxisAlignment: MainAxisAlignment.spaceBetween,
          children: [
            CustomText(
              text: 'Subtotal',
              fontSize: 13.sp,
            ),
            CustomText(
              text: '\${_subtotal.toStringAsFixed(2)}',
              fontSize: 13.sp,
              fontWeight: FontWeight.w600,
            ),
          ],
        ),
        SizedBox(height: 4.h),
        // Final total price after the discount
        Row(
          mainAxisAlignment: MainAxisAlignment.spaceBetween,
          children: [
            CustomText(
              text: 'Discounted total',
              fontSize: 15.sp,
              fontWeight: FontWeight.bold,
            ),
            CustomText(
              text: '\${_discountedSubtotal.toStringAsFixed(2)}',
              fontSize: 16.sp,
              fontWeight: FontWeight.bold,
              color: theme.colorScheme.primary,
            ),
          ],
        ),
      ],
    ),
  );
}
```



```
// skeleton loading (tama ba tong term na to hahaha?)
Widget _buildSkeletonList(bool isDark) {
  return ListView.separated(
    padding: EdgeInsets.symmetric(horizontal: 16.w, vertical: 16.h),
    itemCount: 5,
    separatorBuilder: (context, index) => SizedBox(height: 12.h),
    itemBuilder: (context, index) {
      return Container(
        padding: EdgeInsets.all(10.r),
        decoration: BoxDecoration(
          color: isDark ? AppColors.darkCard : AppColors.lightCard,
          borderRadius: BorderRadius.circular(16.r),
          border: Border.all(
            color: isDark ? AppColors.darkBorder : AppColors.lightBorder,
          ),
        ),
        child: Row(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            // container for the item thumbnail
            Container(
              width: 80.w,
              height: 80.h,
              decoration: BoxDecoration(
                color:
                  isDark ? AppColors.darkSkeleton : AppColors.lightSkeleton,
                borderRadius: BorderRadius.circular(12.r),
              ),
            ),
            SizedBox(width: 12.w),
            Expanded(
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.start,
                children: [
                  // container for the item title
                  Container(
                    width: 140.w,
                    height: 12.h,
                    color:
                      isDark ? AppColors.darkBorder : AppColors.lightBorder,
                  ),
                  SizedBox(height: 8.h),
                  // container for the price row
                  Container(
                    width: 90.w,
                    height: 12.h,
                    color:
                      isDark ? AppColors.darkBorder : AppColors.lightBorder,
                  ),
                  SizedBox(height: 8.h),
                  // container for the discount row
                  Container(
                    width: 60.w,
                    height: 12.h,
                    color:
                      isDark ? AppColors.darkBorder : AppColors.lightBorder,
                  ),
                ],
              ),
            ),
          ],
        ),
      );
    },
  );
}
```



homescreen.dart

```
import 'package:flutter/material.dart';
import 'package:flutter_screenutil/flutter_screenutil.dart';

import 'cart_screen.dart';
import 'product_screen.dart';
import '../constants.dart';
import '../widgets/custom_text.dart';

class HomeScreen extends StatefulWidget {
  final String username;

  const HomeScreen({super.key, this.username = ''});

  @override
  State<HomeScreen> createState() => _HomeScreenState();
}

class _HomeScreenState extends State<HomeScreen> {
  int _selectedIndex = 0;
  final PageController _pageController = PageController();

  // cart page index saved in a variable.
  static const int _cartPageIndex = 1;

  @override
  void dispose() {
    _pageController.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    // Read the current color theme settings
    final theme = Theme.of(context);
    final isDark = theme.brightness == Brightness.dark;
    final onAppBar = theme.appBarTheme.foregroundColor ?? Colors.white;

    return PopScope(
      canPop: false,
      child: Scaffold(
        appBar: AppBar(
          automaticallyImplyLeading: false,
          elevation: 0,
          title: _selectedIndex == 0
            ? Image.asset(
                'assets/images/nubdexchange_logo.png',
                scale: 11.5.sp,
              )
            : CustomText(
                text: _selectedIndex == _cartPageIndex ? 'Cart' : 'Profile',
                fontSize: 20.sp,
                fontWeight: FontWeight.w600,
                color: onAppBar,
              ),
          actions: [
            IconButton(
              icon: Icon(Icons.settings, size: 24.sp, color: onAppBar),
              onPressed: () => Navigator.pushNamed(context, '/settings'),
            ),
          ],
        ),
        body: PageView(
          physics: const NeverScrollableScrollPhysics(),
          controller: _pageController,
          children: const <Widget>[
            ProductScreen(),
            CartScreen(),
            SizedBox.expand(),
          ],
          onPageChanged: (page) {
            setState(() {
              _selectedIndex = page;
            });
          },
        ),
      ),
    );

    //chat icon navigation -> floating icon
    floatingActionButton: _selectedIndex == _cartPageIndex
      ? null
      : FloatingActionButton(
          backgroundColor: theme.colorScheme.primary,
          onPressed: _onChatPressed,
          tooltip: 'Chat',
          child: Icon(
            Icons.chat,
            size: 24.sp,
            color: theme.colorScheme.onPrimary,
          ),
        ),
    bottomNavigationBar: BottomNavigationBar(
      showSelectedLabels: true,
      showUnselectedLabels: false,
      selectedItemColor: theme.colorScheme.primary,
      unselectedItemColor:
        isDark ? AppColors.darkMuted : AppColors.lightMuted,
      backgroundColor: isDark ? AppColors.darkCard : AppColors.lightCard,
      type: BottomNavigationBarType.fixed,
      onTap: _onTappedBar,
      items: const [
        BottomNavigationBarItem(
          icon: Icon(Icons.shop_2),
          label: 'Shop',
        ),
        BottomNavigationBarItem(
          icon: Icon(Icons.shopping_cart),
          label: 'Cart',
        ),
        BottomNavigationBarItem(
          icon: Icon(Icons.person),
          label: 'Profile',
        ),
      ],
      currentIndex: _selectedIndex,
    ),
  );
}

// for chat icon on press function (wala pa function for now)
void _onChatPressed() {
}

void _onTappedBar(int value) {
  setState(() {
    _selectedIndex = value;
  });
  _pageController.jumpToPage(value);
}
```



product_detail_screen.dart

```
// the bar at the bottom that has the Add to Cart button
Widget _buildAddToCartBar(Product product, ThemeData theme, bool isDark) {
  // We cannot add a product that has no stock left
  final isOutOfStock = product.stock <= 0;

  return Container(
    padding: EdgeInsets.symmetric(horizontal: 16.w, vertical: 12.h),
    decoration: BoxDecoration(
      color: theme.cardTheme.color,
      border: Border(
        top: BorderSide(
          color: isDark ? AppColors.darkBorder : AppColors.lightBorder,
        ), // BorderSide
      ), // Border
    ), // BoxDecoration
    child: SafeArea(
      top: false,
      child: SizedBox(
        width: double.infinity,
        height: 48.h,
        //add to cart btn
        child: ElevatedButton.icon(
          onPressed: () {},
          icon: Icon(Icons.add_shopping_cart, size: 20.sp),
          label: CustomText(
            text: isOutOfStock ? 'Out of Stock' : 'Add to Cart',
            fontSize: 15.sp,
            fontWeight: FontWeight.bold,
          ), // CustomText
          style: ElevatedButton.styleFrom(
            shape: RoundedRectangleBorder(
              borderRadius: BorderRadius.circular(12.r),
            ), // RoundedRectangleBorder
          ),
        ), // ElevatedButton.icon
      ), // SizedBox
    ), // SafeArea
  ); // Container
}
```

cart_service.dart

```
import 'dart:convert';
import 'package:http/http.dart' as http;
import '../constants.dart';
import '../models/cart.dart';

class CartService {
  Future<List<Cart>> getAllCarts() async {
    final response = await http.get(Uri.parse('$host/carts'));

    if (response.statusCode == 200) {
      final Map<String, dynamic> data = jsonDecode(response.body);
      final List cartsJson = data['carts'] ?? [];
      return cartsJson.map((json) => Cart.fromJson(json)).toList();
    } else {
      throw Exception('Failed to load carts');
    }
  }

  // Call API to get the carts of one user only
  // GET https://dummyjson.com/carts/user/{userId}
  Future<List<Cart>> getCartsByUserId(int userId) async {
    // user id
    final response = await http.get(Uri.parse('$host/carts/user/$userId'));

    if (response.statusCode != 200) {
      throw Exception('Failed to load user cart');
    }

    // JSON text -> Map
    final Map<String, dynamic> data = jsonDecode(response.body);
    final List cartsJson = data['carts'] ?? [];
    return cartsJson.map((json) => Cart.fromJson(json)).toList();
  }

  // Call API to add a product to a cart
  // POST https://dummyjson.com/carts/add
  Future<Cart> addToCart({
    required int userId,
    required int productId,
    required int quantity,
  }) async {
    //sending data to the API
    final response = await http.post(
      Uri.parse('$host/carts/add'),
      headers: {'Content-Type': 'application/json'},
      body: jsonEncode({
        'userId': userId,
        'products': [
          {
            'id': productId,
            'quantity': quantity,
          }
        ],
      })),
    );

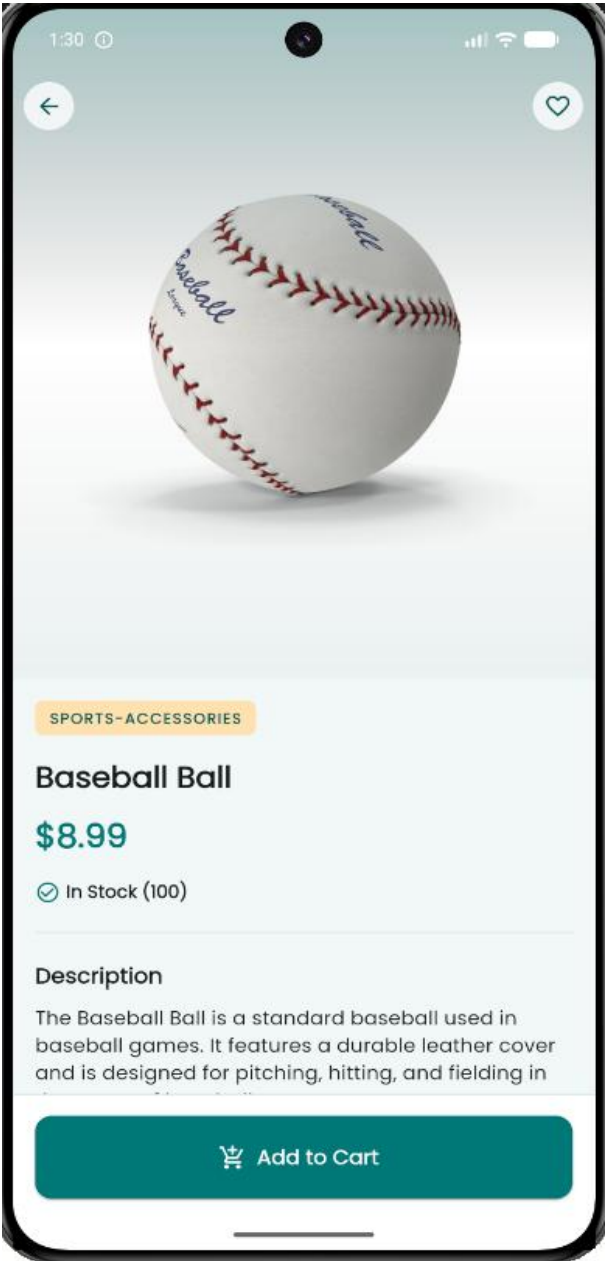
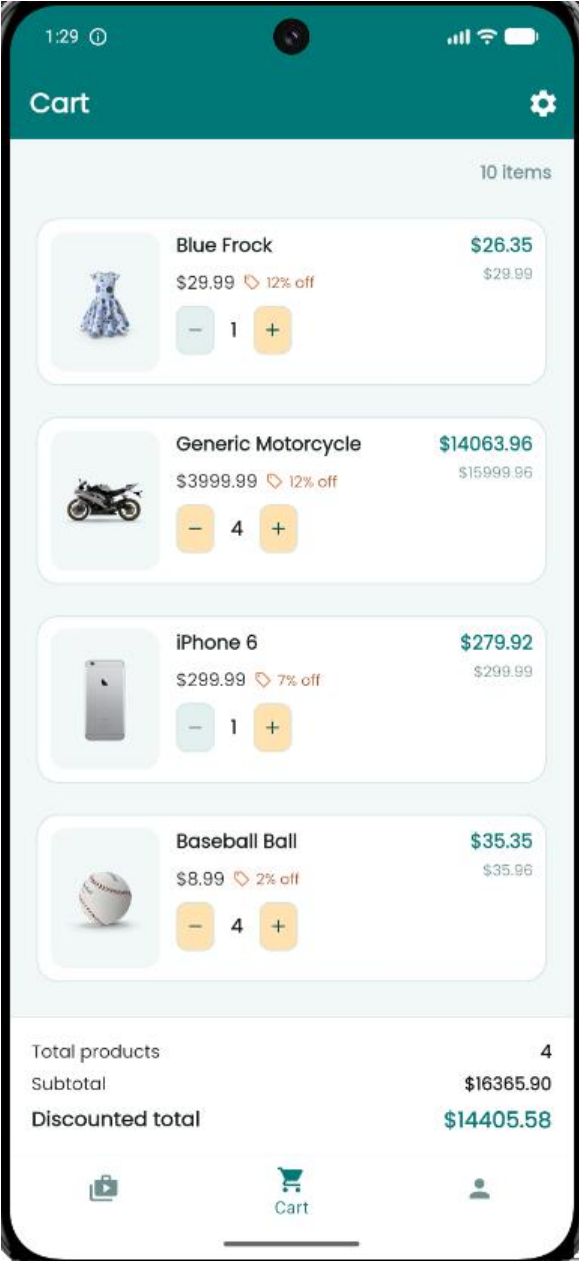
    if (response.statusCode != 200 && response.statusCode != 201) {
      throw Exception('Failed to add product to cart');
    }

    final Map<String, dynamic> data = jsonDecode(response.body);
    return Cart.fromJson(data);
  }
}
```

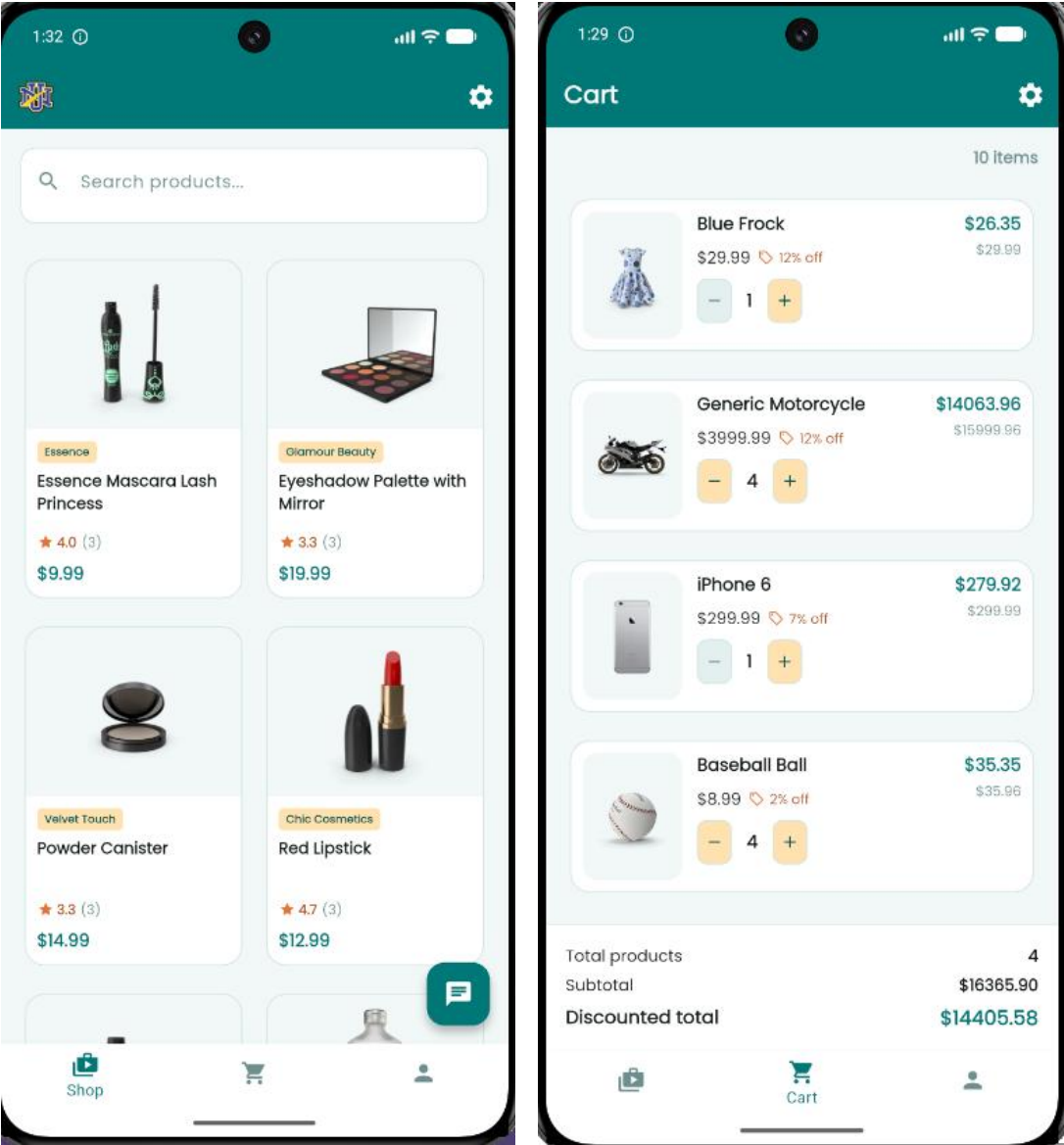


Output – [name of the output]

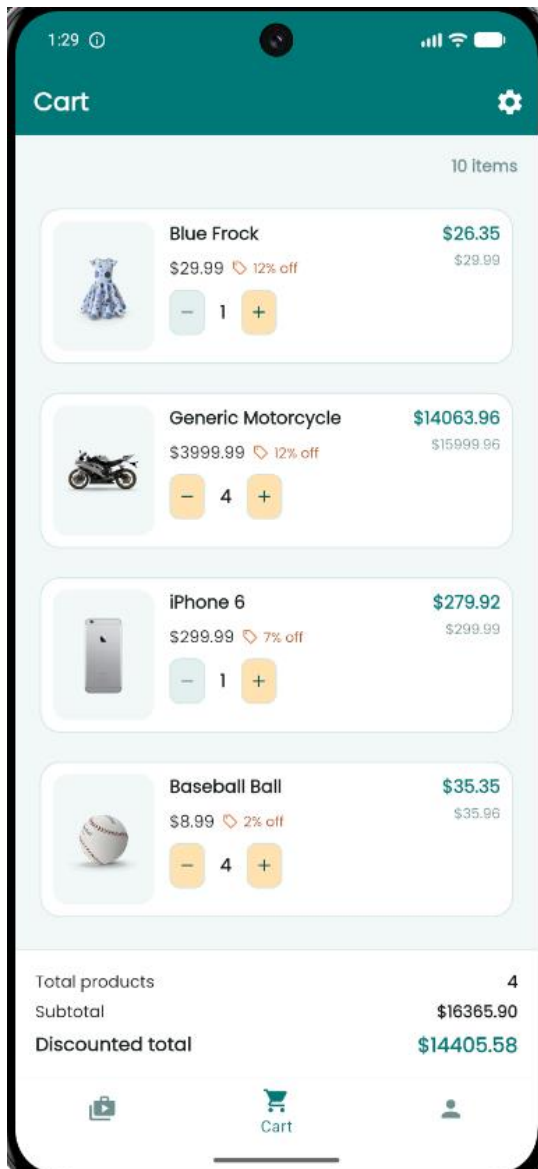
Enhancement 1: Make a cart_screen in order to render the new API endpoint. The items on the cart_screen must be clickable going to the detail_screen for the utilization of the screen widget.



Enhancement 2: Make the chat bottom navigation as FloatingActionButton. When in the cart_screen the FloatingActionButton must be hidden.



Enhancement 3: Read the Cart documentation <https://dummyjson.com/docs/carts> and check how to integrate cart by user id. To render only one user cart. Also try to use add to cart by passing the values of the product => cart <https://dummyjson.com/carts/add>



Laboratory Discussion: Discuss how the cart model, services and screen interact with each other to render the API endpoint going to the same `detail_screen.dart`. Discuss the updated design pattern in this activity. Also discuss how to use `getbyId` at Cart endpoint.

The three cart files each have their own function:

- **cart.dart** turns the JSON data into Cart and CartProduct objects.
- **cart_service.dart** gets the cart data from GET `/carts/user/1`, reads the "carts" list, and converts it using `Cart.fromJson()`.
- **cart_screen.dart** calls the service when the screen opens, saves the result, and shows the loading, error, empty, or cart list screen.

This keeps the same Screen → Service → Model structure: the screen doesn't handle the API URL, and the service doesn't handle the UI.

For the product details, the cart only passes the `CartProduct.id` to **detail_screen.dart**, and the detail screen uses that ID to get the full product. This allows both the product screen and cart to use the same detail screen. The cart total is calculated from the items because the user can change the quantity, which can make the API's original total outdated. `CartProduct` now allows quantity, total, and `discountedTotal` to change through `updateQuantity()`. This is a simple shortcut for now; in a bigger app, this changing data would usually be handled by a provider. Lastly, `getCartById` is simpler because GET `/carts/1` gives one cart directly, so there is no need for the "carts" list or `.map()`. A cart ID is also different from a user ID—they only happen to look the same in DummyJSON's sample data. DummyJSON's sample data.

Lab Activity Rubric {*Copy and paste the Rubric here*}

Name	Jamaine Grace Tuazon
Section	INF231
Date	August 20, 2026

Criteria	Excellent (4 pts)	Good (3 pts)	Fair (2 pts)	Poor (1 pt)	Points
Code Implementation	Code runs flawlessly, includes all required features (placeholders, avatars, widgets for like/comment).	Code mostly correct, minor errors, or missing one minor feature.	Noticeable errors or missing two or more features.	Code does not run or is significantly incomplete.	
Design Consistency	UI closely resembles intended layout, proper spacing, alignment, and styling.	Layout mostly accurate, minor spacing or alignment issues.	Layout deviates significantly from the intended design or appears incomplete.	Layout is unrecognizable or does not follow the instructions.	
Use of Widgets and Enhancements	Properly implements widgets for placeholders, avatars, and like/comment buttons.	Most widgets are correctly implemented, with minor issues or missing one enhancement.	Widgets are inconsistently used or missing two or more enhancements.	Widgets are not used, or enhancements are not implemented.	
Code Organization and Readability	Code is well-structured, follows best practices, and adheres to the specified file organization.	Code is mostly organized and readable, minor deviations from the required structure.	Code is poorly organized or does not follow the specified structure.	Code is disorganized and difficult to understand.	
Back-End Integration	Back-end integration works flawlessly with efficient API calls, secure data handling, and real-time updates; no errors or downtime.	Back-end integration is mostly correct with only minor API, data, or response handling issues.	Back-end integration has noticeable flaws or inefficiencies; multiple API calls fail or data handling is inconsistent.	Back-end integration has major issues; frequent failures, poor response handling, or insecure practices.	
Other Comments/ Observations:				Total Score	
				Rating: (Total Score/20) * 100	